Name: Timothy Tan Zhi-Jing
Github repo link: https://github.com/tlmotan/approval-workflow-shortcut
Google Drive link:
https://drive.google.com/file/d/11Xwss2Sr3CzKXIznd9LTPe4b2Kiev5Wz/view?usp=sharing

How I planned and approached the app

Before implementing the app, I needed to come up with an idea. I filtered the candidate ideas using these three simple criterias:

1. **Is it a problem that people face everyday?** This is to make sure it is grounded in real needs.
2. **Who are the people facing this problem?** This is so that I can gauge what the design and user experience should be like. For example, if more technical people are using the platform, the design should be less flashy and the user experience should be more straightforward.
3. Lastly, **can it be solved with software?**

After coming up with the idea, I needed to define the two core features that will make the solution feasible. This is to remove clutter and prevent unnecessary complexity.

After that, I had to decide on the tech stack to bring this to fruition.

Why I chose certain tools, frameworks, and libraries

Next.js (App Router) + React + TypeScript

I chose this framework because it lets the API routes and UI share types end-to-end, reducing the chance of a mismatch between what the server enforces and what the client assumes.

SQLite + Prisma

- **SQLite** is file-based and needs zero setup. Given the time constraint, I prioritized the correctness of the state-machine logic over the infrastructure of the project, so I didn't want to spend hours building on database provisioning.
- **Prisma's declarative schema** is what lets a single @@unique([claimId, stageIndex]) constraint carry real weight: the “two approvers act on the same stage at once” race condition is closed at the database level, rather than depending on application code getting a check-then-write sequence exactly right under concurrency. That one line is arguably the most load-bearing part of the schema.

No authentication layer

Users are selected via a seeded “Acting as” picker (name + role) rather than a login system. This was a deliberate scope decision, not a shortcut.

Authentication contributes little to the correctness problem being demonstrated, and building it would trade hours away from the state machine, tests, and documentation. Which are the parts actually being evaluated. In a production version, this picker would be replaced by real authenticated sessions.

Main technical decisions and the reasoning behind them

- **Status is derived, never stored.** There is no mutable status column. “Approved” is computed from the full ApprovalRecord history every time it's read: reachable if and only if every stage in the chain has an approved record, in order, and no rejected record exists anywhere in that claim's history. This makes an invalid status structurally unreachable, rather than merely unlikely.

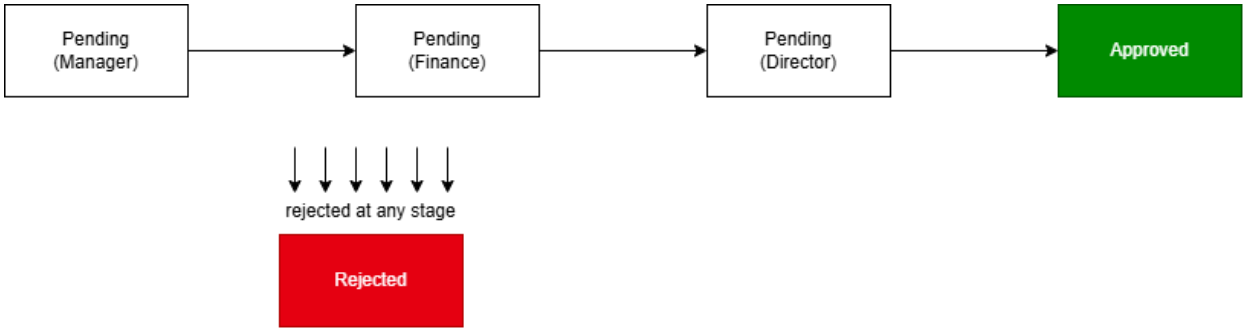
- **The approval chain is locked at submission, not recomputed live.** Chain length depends on claim amount. I considered recomputing it against current policy at each stage, but rejected that: retroactively changing an in-progress state machine's structure is exactly where correctness bugs live (e.g. inserting a new required stage after some stages have already been decided is ambiguous to resolve correctly). Locking it accepts that an in-flight claim may run under slightly stale policy, in exchange for a guarantee that its structure can never become inconsistent.

- **Enforcement is role-based; attribution is name-based.** The chain specifies required roles (Manager, Finance, Director), and any employee holding that role may act at that stage, mirroring how real approval chains work. Each ApprovalRecord still stores the acting employee's actual name at write time (denormalized, not just joined live), so the audit trail remains accurate even if an employee's role changes later.

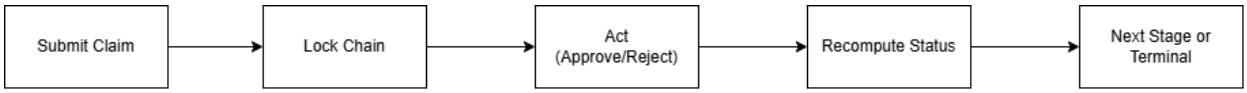
- **Rejection is absorbing, not recoverable.** A rejected claim cannot be reopened. A resubmission creates a new ExpenseClaim row (linked via resubmittedFrom for traceability) rather than mutating the old one, the same pattern as a brokerage app showing a fresh order after a failed one, so no history is ever overwritten.

Flowchart 1: Approval state diagram

Rejected is reachable as an absorbing state from any pending stage; once reached, no further transitions are possible.



Flowchart 2: Claim lifecycle



Architecture overview and tech stack

- Frontend & API: Next.js (App Router) + React + TypeScript, UI and API routes in one codebase, to share types.
- Core logic: getClaimStatus() and act() are pure, testable functions, kept independent of the HTTP layer so they can be unit-tested directly.
- Data layer: Prisma ORM over SQLite; Employee, ExpenseClaim, and ApprovalRecord models, with the unique(claimId, stageIndex) constraint enforcing one decision per stage at the database level.
- No external services: no auth provider, no email/notification service, kept deliberately out of scope (see technical decisions above).

Where I used AI, and how I checked its output

- **Design reasoning:** I used Claude to pressure-test my state-machine design before writing code, walking through the data model, the chain-locking decision, and the concurrency edge case out loud, in a quiz format, so I had to explain each choice in my own words rather than accept a suggestion passively.
- **Caught and corrected:** An early draft of my data model stored the full list of approvers-so-far inside a single ApprovalRecord. Talking it through, I identified this collapsed two distinct concerns (the plan vs. the history) into one, and corrected it to an append-only log of individual decisions instead, one row per actual decision.
- **Implementation scaffolding:** I used Claude Code with a project-level CLAUDE.md documenting the invariant, data model, and required test cases, so generated code stays anchored to a specification I wrote and understood first, rather than the AI inventing scope.
- **Verification:** Every design decision above is backed by one of seven required test cases (valid chains, out-of-order attempts, wrong-role attempts, double-decision race conditions, rejection-then-resubmission) written against the pure logic functions before the UI was built, so correctness could be checked independently of how the app looks.